

Operating Systems Laboratory 2 Report

Working with Processes

Skanda P (CS24BT045)

Narapogu Suhas (CS24BT019)

1 Objective

This assignment explores Linux process creation and management using `fork()`, `execl()`, `wait()`, `waitpid()`, `getpid()`, and signals (`kill()`, `signal()`) while searching a large text file.

- **Part I** performs a single-process, single-range search.
- **Part II** recursively partitions the range into a binary process tree and always waits for both children, even after a match is found.
- **Part III** builds the same tree but sends `SIGTERM` to a still-running sibling after success, cascading the termination through unnecessary descendants.

The pattern is `NGTNIJGK`; `file.txt` is 67,108,864 bytes; the range is `[0,67108863]`; and the Part II/III chunk limit is 8,388,608 bytes. All parts find the pattern at **position 64520807**.

2 Part I — Single-Process Search

`part1_searcher.cpp` opens the file, seeks to the start position, and reads the range. It then calls `std::string::find`, prints the PID and result, and returns 1 on success or 0 otherwise.

Command:

```
./part1.out file.txt NGTNIJGK 0 67108863
```

Output:

```
[469] found at 64520807
```

One process searches the full 64 MiB range and finds the pattern at byte 64520807.

3 Part II — Recursive Partitioning Without Killing

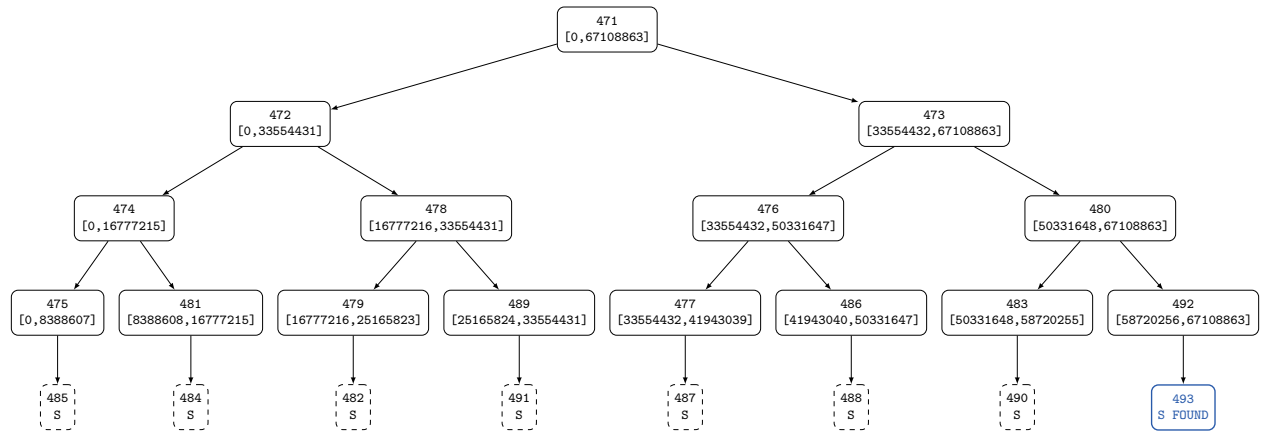
`part2_partitioner.cpp` starts one `part2_searcher.out` when its range is at most 8,388,608 bytes. Larger ranges are split at the midpoint and executed by left and right `part2_partitioner.out` children. The parent calls `waitpid()` for the left child and then the right child, so both branches finish even if one reports success early.

Command:

```
./part2_partitioner.out file.txt NGTNIJGK 0 67108863 8388608
```

Three split levels divide the 2^{26} -byte range into eight chunks, producing 15 partitioners and 8 searcher leaves: 23 processes total.

3.1 Process tree



Solid boxes are partitioners (PID and range); dashed “S” boxes are searchers. PID 493 finds the pattern at 64520807, while every other branch finishes naturally.

3.2 Order in which processes were created

The captured creation order is below (L/R = left/right partitioner; S = searcher):

1. 471: root, [0,67108863]
2. 472: L of 471, [0,33554431]
3. 473: R of 471, [33554432,67108863]
4. 474: L of 472, [0,16777215]
5. 475: L of 474, [0,8388607]
6. 476: L of 473, [33554432,50331647]
7. 477: L of 476, [33554432,41943039]
8. 478: R of 472, [16777216,33554431]
9. 479: L of 478, [16777216,25165823]
10. 480: R of 473, [50331648,67108863]
11. 481: R of 474, [8388608,16777215]
12. 482: S of 479
13. 483: L of 480, [50331648,58720255]
14. 484: S of 481
15. 485: S of 475
16. 486: R of 476, [41943040,50331647]
17. 487: S of 477
18. 488: S of 486
19. 489: R of 478, [25165824,33554431]
20. 490: S of 483
21. 491: S of 489
22. 492: R of 480, [58720256,67108863]
23. 493: S of 492 (**found**)

3.3 Order in which processes finished

The captured output/return order is listed below. NF means “didn’t find”; S/L/R followed by a PID means that searcher/left/right child returned.

1. 484: NF
2. 481: S484 returned
3. 482: NF
4. 485: NF
5. 475: S485 returned
6. 474: L475 returned

7. 474: R481 returned
8. 479: S482 returned
9. 488: NF
10. 486: S488 returned
11. 487: NF
12. 477: S487 returned
13. 476: L477 returned
14. 476: R486 returned
15. 490: NF
16. 483: S490 returned
17. **493: found at 64520807**
18. 492: S493 returned
19. 480: L483 returned
20. 480: R492 returned
21. 473: L476 returned
22. 473: R480 returned
23. 491: NF
24. 489: S491 returned
25. 478: L479 returned
26. 478: R489 returned
27. 472: L474 returned
28. 472: R478 returned
29. 471: L472 returned
30. 471: R473 returned; root exits 1

PID 493 reports success at step 17, but 13 further return events occur because Part II never terminates unnecessary work.

4 Part III — Killing Processes Once the Pattern Is Found

Part III builds the same tree, but every process first installs a `SIGTERM` handler. The handler prints the PID, forwards the signal to its children, reaps them with `waitpid()`, and calls `_exit(0)`, preventing zombies and orphans.

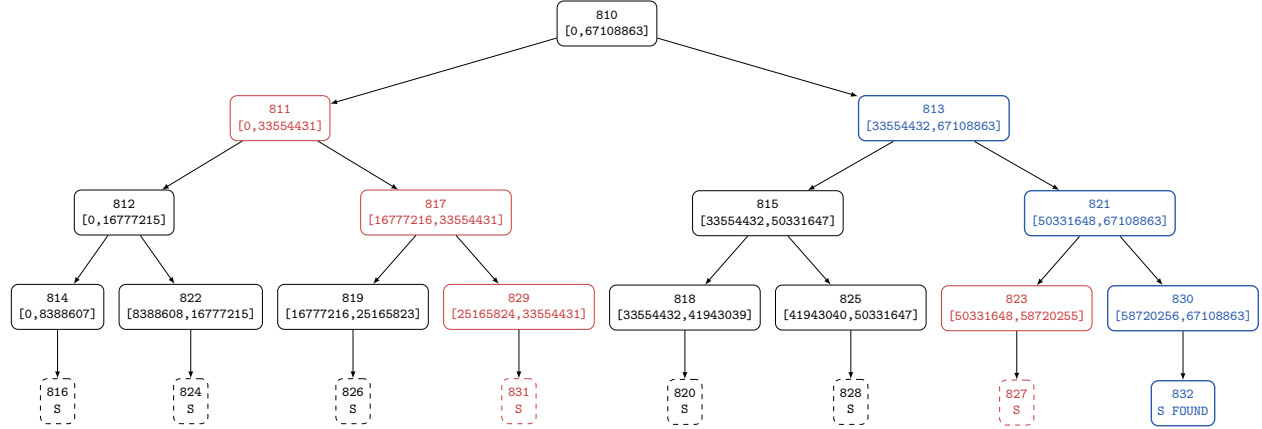
A partitioner uses `wait()` to collect whichever child exits first. If that child returns 1, it terminates and reaps the still-running sibling and returns 1; otherwise it waits for the second child. Since handlers forward `SIGTERM`, one `kill()` cascades through the entire unnecessary subtree.

Command:

```
./part3_partitioner.out file.txt NGTNIJGK 0 67108863 8388608
```

The tree still contains 15 partitioners and 8 searchers. In this run the right-most leaf wins.

4.1 Process tree



Blue boxes form the winning path $810 \rightarrow 813 \rightarrow 821 \rightarrow 830 \rightarrow 832$. Red boxes are terminated directly or through a signal cascade. PID 832 finds the pattern at 64520807.

4.2 Order in which processes were created

The observed creation order was:

- | | |
|--|--|
| 1. 810: root, [0,67108863] | 13. 822: R of 812, [8388608,16777215] |
| 2. 811: L of 810, [0,33554431] | 14. 823: L of 821, [50331648,58720255] |
| 3. 813: R of 810, [33554432,67108863] | 15. 824: S of 822 |
| 4. 812: L of 811, [0,16777215] | 16. 825: R of 815, [41943040,50331647] |
| 5. 814: L of 812, [0,8388607] | 17. 826: S of 819 |
| 6. 815: L of 813, [33554432,50331647] | 18. 827: S of 823 |
| 7. 816: S of 814 | 19. 828: S of 825 |
| 8. 817: R of 811, [16777216,33554431] | 20. 829: R of 817, [25165824,33554431] |
| 9. 818: L of 815, [33554432,41943039] | 21. 830: R of 821, [58720256,67108863] |
| 10. 819: L of 817, [16777216,25165823] | 22. 831: S of 829 |
| 11. 820: S of 818 | 23. 832: S of 830 (found) |
| 12. 821: R of 813, [50331648,67108863] | |

4.3 Order in which the pattern was found and processes were killed

With `std::unitbuf` flushing each line, the captured real-time order was (NF and S/L/R use the same notation as Part II; TERM means `SIGTERM` received):

1. 816: NF
2. 814: S816 returned
3. 812: L814 returned
4. 827: NF
5. 820: NF
6. 826: NF
7. 819: S826 returned
8. 817: L819 returned
9. 818: S820 returned
10. 815: L818 returned
11. **832: found at 64520807**
12. 830: S832 returned; return 1
13. **821: R830 returned 1; TERM/reap 823; return 1**
14. **823: TERM; 827 had exited but was not yet processed**
15. 824: NF
16. 822: S824 returned

17. 812: R822 returned
18. 811: L812 returned
19. 828: NF
20. 825: S828 returned
21. 815: R825 returned
22. 813: L815 returned
23. **813: R821 returned 1; return 1**
24. **810: R813 returned 1; TERM/reap 811; exit 1**
25. **811: TERM → 817**
26. **817: TERM → 829**
27. **829: TERM → 831**
28. **831: TERM; leaf exits**

4.4 Observations

- Branches already finished when success reaches an ancestor need no signal; only live siblings are terminated.
- One `kill(811, SIGTERM)` cascades $811 \rightarrow 817 \rightarrow 829 \rightarrow 831$ because each handler knows its own children; the root need not know the leaf PIDs.
- PID 823 demonstrates a scheduling race: child 827 had exited, but 823 received `SIGTERM` before it could report/reap that completion.
- Part II always completes all 23 processes. Part III depends on scheduling: here six searchers finished wasted work, while 831 was terminated before searching.

5 Conclusion

The lab used `fork()`, `execl()`, `wait()/waitpid()`, `getpid()`, and POSIX signals to progress from a single searcher to a binary process tree. Part II exposes the cost of waiting for all work; Part III propagates `SIGTERM` through unnecessary branches and reaps every child, reducing wasted work without leaving zombies.